

This assignment has in total 100 base points and 10 extra points, and the cap is 100. Bonus questions are indicated using the ★ mark.

Submission Instructions: Please submit to Gradescope. During submission, you need to **mark/map the solution to each question**; otherwise, we may apply a penalty.

Another notice: you only get full credits for the algorithm design questions if your algorithm matches the desirable complexity. If the desirable complexity is not stated, you need to design an algorithm to be as fast as possible.

Please specify the following information before submission:

- Your Name:
- Your NetID:

Problem 1: Packing heavy unit intervals [15 pts]

Suppose we are given a set $P = \{x_1, \dots, x_n\}$ of n points on the x -axis and a positive integer k . We say an interval on the x -axis is k -heavy if it contains at least k points in P . We consider *left-open right-closed unit intervals*, i.e., intervals of the form $I = (a, b]$ where $b - a = 1$ (for convenience, below we simply call them *unit intervals*). Our goal is to find a maximum number of k -heavy unit intervals on the x -axis that are *disjoint* from each other. For example, if $P = \{-\frac{2}{3}, 0, \frac{2}{3}\}$ and $k = 1$, then one optimal solution is $I_1 = (-1.6, -0.6]$, $I_2 = (-0.6, 0.4]$, $I_3 = (0.5, 1.5]$, which consists of three disjoint k -heavy unit intervals (note that the optimal solution is not necessarily unique). Design a greedy algorithm $\text{HEAVYINT}(n, P, k)$ to solve this problem, which should output the set of intervals you find. First describe your greedy strategy and prove its correctness. Then implement your algorithm by giving the pseudocode. Your algorithm is supposed to run in $O(n)$ time, assuming the points in P have already been sorted in the left-right order.

Solution.

- Only at points $a = x_i$ or $a = x_i - 1$ (for $x_i \in P$) does the count in $(a, a + 1]$ change. Thus, define

$$Q = \{x_i - 1 : x_i \in P\} \cup \{x_i : x_i \in P\}.$$

- **Greedy:** Initialize $L = \min(P) - 1$ (the minimum allowed left endpoint) and scan Q for the smallest candidate $a \geq L$ such that the count of points in $(a, a + 1]$ is at least k . Once such an a is found, choose that interval, update L to $a + 1$, and repeat.
- **Correctness:** By always choosing the leftmost feasible candidate:
 - Each interval is heavy (i.e. contains at least k points).
 - Intervals do not overlap (since the next interval's left endpoint is at least the previous one's right endpoint).
 - An exchange argument shows that shifting intervals left does not reduce their heavy property, ensuring this greedy strategy is optimal.

```

1  HEAVY-INT(P, k)
2      Input: sorted P = [x1, x2, ..., xn], positive integer k
3      Output: set S of disjoint heavy unit intervals (a, a+1]
4
5      S <- {}                                // Solution set
6      L <- min(P) - 1                        // Allowed left endpoint
7      Q <- merge({x - 1 for x in P}, {x for x in P})
8                                          // Q is sorted and has at most 2n candidates

```

```
9
10     p_left <- 1, p_right <- 1
11
12     // Find first candidate a_star >= L with count >= k
13     for each a_candidate in Q where a_candidate >= L do
14         while p_left <= n and P[p_left] <= a_candidate do
15             p_left <- p_left + 1
16         while p_right <= n and P[p_right] <= a_candidate + 1 do
17             p_right <- p_right + 1
18         if (p_right - p_left) >= k then
19             a_star <- a_candidate
20             break
21         end if
22     end for
23     if no candidate found then
24         return S
25
26     // Greedily select intervals
27     while true do
28         S <- S ∪ { (a_star, a_star+1] }
29         L <- a_star + 1
30         Advance candidate pointer in Q to first a_candidate >= L
31         found <- false
32         for each a_candidate in Q where a_candidate >= L do
33             Update p_left, p_right for a_candidate
34             if (p_right - p_left) >= k then
35                 a_star <- a_candidate
36                 found <- true
37                 break
38             end if
39         end for
40         if not found then
41             break
42     end while
43     return S
```

Problem 2: Sorting with arbitrary swaps [10 + 10 + 10* pts]

Given an array $A[1 \dots n]$ of distinct numbers, we want to sort it in increasing order. However, we are only allowed to change A by swapping two elements (not necessarily adjacent) in A . Specifically, we are provided an oracle SWAP. If we call $\text{SWAP}(i, j)$, then $A[i]$ and $A[j]$ will be swapped in A .

- (1) Design a simple algorithm that sorts A by calling the SWAP oracle at most $n - 1$ times. Describe the basic idea and give the pseudocode, and analyze the number of oracle calls. Your algorithm is supposed to run in $O(n \log n)$ time (and call SWAP at most $n - 1$ times), assuming each oracle call takes $O(1)$ time.
- (2) Next, we consider the problem of sorting A using *minimum* number of calls of the SWAP oracle. Prof. Greed gives the following greedy algorithm for this problem. Let $\#\text{Inv}(A)$ denote the number of inversions in A . The algorithm simply repeats the following step until A is sorted: find indices $i, j \in \{1, \dots, n\}$ such that swapping $A[i]$ and $A[j]$ makes $\#\text{Inv}(A)$ decrease the most, and then call $\text{SWAP}(i, j)$. Give a counterexample for which this algorithm fails to give an optimal solution (and briefly argue why it is a counterexample). To get the full credit, your example should make the algorithm fail no matter how it breaks the ties.
- (c*) Design an algorithm that sorts A using minimum number of SWAP calls. Describe the basic idea and give the pseudocode. Prove the correctness of your algorithm (i.e., it successfully sorts A and the number of SWAP calls used is minimum). Your algorithm is supposed to run in $O(n^2)$ time or faster.

Solution. Please write down your solution to Problem 2 here.

Problem 3: Longest doubling subsequence [15 pts]

Recall that an array (or sequence) A of real numbers is *doubling* if $A[i] \geq 2A[i-1]$ for all $i \geq 2$ (thus any array of length 1 is doubling). Given an array A , we want to find a longest subsequence of A that is doubling. For example, if $A = [7, 1, 3, 8, 2, 4, 5, 6, 9]$, then a longest doubling subsequence of A is $[1, 2, 4, 9]$, which is of length 4. Design an algorithm $\text{LDS}(n, A)$ that returns a longest doubling subsequence A' of the array A (where n is the size of A). Describe the basic idea of your algorithm and give the pseudocode. Briefly justify its correctness and analyze its time complexity. Your algorithm should run in $O(n^2)$ time or faster.

Solution. **Please write down your solution to Problem 3 here.**

Problem 4: Removing the numbers optimally [20 pts]

Given a sequence of (positive) numbers, we want to remove the numbers from the sequence one by one. When removing one number x , we gain a score equal to l^2xr^2 where l is the number to the left of x in the current sequence ($l = 1$ if x is the leftmost number in the current sequence) and r is the number to the right of x in the current sequence ($r = 1$ if x is the rightmost number in the current sequence). For example, suppose the given sequence is $(2, 9, 12, 3)$. If we remove 12 first, the score we gain is $9^2 \times 12 \times 3^2 = 8748$, and the sequence becomes $(2, 9, 3)$. Now if we remove 9 from the sequence, then the score is $2^2 \times 9 \times 3^2 = 324$, and the sequence becomes $(2, 3)$. Next if we remove 3, then the score is $2^2 \times 3 \times 1^2 = 12$, and the sequence becomes (2) . Finally we remove 2, and the score is $1^2 \times 2 \times 1^2 = 2$. The total score we gain is $8748 + 324 + 12 + 2 = 9086$. Our goal is to find an order to remove the numbers from the given sequence such that the total score we gain is maximized. Formally, design an algorithm $\text{REMOVE}(n, A)$ where A is an array of n positive numbers; the algorithm returns the maximum total score we can gain if the given sequence is A (for convenience, you are not required to return the optimal order for removing the numbers). Describe the basic idea of your algorithm and give the pseudocode. Briefly justify its correctness and analyze its time complexity. Your algorithm should run in $O(n^3)$ time or faster.

Solution. Please write down your solution to Problem 4 here.

Problem 5: Tutorial of Fast Fourier Transform [10 pts]

This is not really a problem but more like a tutorial. Fast Fourier transform is arguably the most important algorithm in signal processing and has enormous applications in various fields (EE, CS, optics, acoustics, etc). Please watch the video from <https://www.youtube.com/watch?v=spUNpyF58BY> about Fourier transform. Even if you are familiar with the concept, the visualizations in the video are unparalleled. It's also highly recommended that you watch other videos from that content maker. Based on the video, answer the following questions:

- (1) Continuous Fourier transform.
 - (i) Considering Fourier transform as a function, what is its domain and range?
 - (ii) Intuitively explain that if the rotation frequency matches some intrinsic frequency of the input signal, the “center of mass” will pop up.
 - (iii) The examples in the video all have the “center of mass” on the x-axis. Should it always be the case?
- (2) Discrete Fourier transform. In practice, we only have discrete samples of a signal (typically evenly spaced). Let $x_1, \dots, x_N$ be the N sequential samples of a continuous signal with duration T . Write out the equation of the discrete Fourier transform that approximates the continuous version. What's the complexity to compute the transform for one frequency value?
- (3) Discrete Fourier transform continued. Since we only have N samples, it is intuitive that the fastest frequency we can assess is N/T and the slowest is $1/T$. We, therefore, only care about the frequencies in the list $1/T, 2/T, 3/T, \dots, N/T$. Rewrite the discrete transforms as a function of k , where the frequency is k/T (for simplicity, we refer to it as frequency k below). What's the complexity if we want to compute the transform for all N frequency values, i.e., we discretize the “center of mass” v.s. frequency figure shown in the video with N evenly-spaced frequency samples as well?
- (4) Symmetry. Considering the previous discrete Fourier transform, compute the equation for frequency $k + N$ and compare it with the frequency k . You may need to use Euler's formula.

We refer to the relationship as a “symmetry”. The following is not directly related to the “symmetry”, but can provide some useful intuitions.

Now, think of the entire transformation into N frequencies as a matrix multiplication. I.e., we have N samples forming a vector as input, and we have N outputs for

the N frequencies. Try to write down a $N \times N$ matrix that performs the transformation.

- (5) Fast Fourier Transform. For frequency k , separate the calculation into two parts, one part containing the x_n 's where n is even and the other part containing the x_n 's where n is odd. Now, consider the even (odd) part as a new transform problem with only half of the samples. Indeed, we are using the idea of **divide-and-conquer**, but how do we save computation? What about the symmetry above? Draft the algorithm to compute discrete Fourier transform in this way and analyze its complexity. For simplicity, you may assume N is a power of 2.

Solution. **Please write down your solution to Problem 5 here.**

Problem 6: Find the common number [5 + 5 pts]

We are given a list of integers $x_1, \dots, x_n$. We know that at least $\lceil n/2 \rceil$ of those numbers have the same value. Unfortunately, you don't have access to the list directly, and the only oracle you have is $\text{COMPARE}(i, j)$, which tells you if x_i and x_j are equal or not.

- (1) Design an oracle that finds the common number. The algorithm has to be deterministic. Analyze the correctness and the complexity in terms of the number of COMPARE .
- (2) What if instead of $n/2$, we call any number that appears at least $\lceil n/k \rceil$ as a common number, where k is a constant integer with value ≥ 2 . We aim to find all the common numbers. You can also directly solve this problem, which generalizes the previous one.

Solution. **Please write down your solution to Problem 6 here.**

Problem 7: Monitoring the statistics [5 pts]

A machine learning student wants to test empirically about sampling Gaussians. The student calls a standard Gaussian sampler at a time, which outputs a real value (e.g., drop one ball down along a very large Galton board). The student then performs sampling repeatedly and wants to keep track of the statistics.

Formally, the student gets x_t coming in a stream, which is generated from a Gaussian sampler. At every time point $t \geq 20$, the student wants to keep a record of three statistics: the 25, 50, and 75 percentiles of all the numbers sampled so far. More precisely, the 25 percentile at time t should return the data point x in $x_1, \dots, x_t$ with rank $\lfloor 0.25 \times t \rfloor$ (similarly for 50 and 75). Suppose the student performs T sampling in total, what's the total complexity of your algorithm?

Solution. **Please write down your solution to Problem 7 here.**

Problem 8: Multi-selection [5 pts]

We are given a list of integers $x_1, \dots, x_n$ (not necessarily sorted). We are also given k distinct integer numbers $1 \leq i_1, \dots, i_k \leq n$ (in sorted order), and we want to find the k elements of $x_1, \dots, x_n$ that have ranks $i_1, \dots, i_k$. Design the algorithm and analyze its correctness and complexity. Another special requirement here is that k is not a constant, so the complexity of k should also be considered.

Solution. **Please write down your solution to Problem 8 here.**